HDOC Day-52

Finding the Previous Greater Element in an Array Using Functions and Pointers in C

Aim

To write a C program using a variable-length integer array, user-defined functions, pointers, and nested loops to find the nearest strictly greater element to the left of every array element. Print -1 when no such element exists, preserving original order and separating results with commas.

Algorithm

1. Start the program and read the array size.
2. Validate that the size is positive, otherwise display an error and terminate.
3. Declare a variable-length integer array arr[size] and read its elements.
4. Pass the array and size to previous_greater() using its address.
5. For each index i, initialize previous to -1.
6. Using an inner loop, scan leftward from i - 1 to index 0.
7. When the first strictly greater element is found, store it and break.
8. Print the result followed by a comma and space except for the last element.
9. Continue until all elements are processed and stop.

C Program

```c
#include <stdio.h>

void previous_greater(int *arr, int size);

int main()
{
    int size, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int arr[size];

    printf("Enter %d array elements: ", size);
    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    previous_greater(arr, size);
    return 0;
}

/* Function to find the previous greater element */
void previous_greater(int *arr, int size)
{
    int i, j, previous;

    printf("\nPrevious Greater Elements: ");
    for (i = 0; i < size; i++)
    {
        previous = -1;

        /* Search elements to the left */
        for (j = i - 1; j >= 0; j--)
        {
            if (*(arr + j) > *(arr + i))
            {
                previous = *(arr + j);
                break;
            }
        }

        printf("%d", previous);
        if (i < size - 1)
        {
            printf(", ");
        }
    }
    printf("\n");
}
```

Explanation

The array is passed as a pointer using previous_greater(arr, size). The expression *(arr + i) accesses arr[i]. For each current element, the inner loop starts at its immediate left neighbor and moves backward. The first value greater than the current element is the nearest previous greater element.

Example

```
Input Array: 4 5 2 10 8
Output: -1, -1, 5, -1, 10
```

Test Case Table

Case	Input Array	Expected Output
1	4 5 2 10 8	-1, -1, 5, -1, 10
2	1 3 2 4	-1, -1, 3, -1
3	5 4 3 2 1	-1, 5, 4, 3, 2
4	1 2 3 4 5	-1, -1, -1, -1, -1
5	2 2 3 3 4	-1, -1, -1, -1, -1
6	10 5 15 20	-1, 10, -1, -1
7	7	-1
8	-3 -1 -2 4	-1, -1, -1, -1
9	size = 0	Invalid array size

Sample Output

```
Enter the size of array: 5
Enter 5 array elements: 4 5 2 10 8

Previous Greater Elements: -1, -1, 5, -1, 10
```

Decreasing Array

```
Enter the size of array: 5
Enter 5 array elements: 5 4 3 2 1

Previous Greater Elements: -1, 5, 4, 3, 2
```

Duplicate Elements

```
Enter the size of array: 5
Enter 5 array elements: 2 2 3 3 4

Previous Greater Elements: -1, -1, -1, -1, -1
```

Invalid Array Size

```
Enter the size of array: 0
Invalid array size!
```

Result

The C program was designed using variable-length arrays, pointers, user-defined functions, nested loops, and conditional statements. The previous_greater() function finds the nearest strictly greater element to the left of each array element. Results appear in original order, separated by commas, with -1 where no such element exists.